RESQAI

INTELLIGENT DISASTER RESPONSE CITIZEN APPLICATION

PROJECT REPORT

Submitted by

2116240701458 – SAISANJAY SS

2116240701069 –BALAJI S



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "**RESQAI Intelligent Disaster Response Citizen Application**" is the bonafide work of "**SAISANJAY SS , BALAJI S**" bearing register number(s) **240701458 , 240701069** who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

ResQAI is an intelligent disaster response citizen application being developed using Flutter and Dart. The project aims to provide a convenient mobile platform through which citizens can access emergency-related features and prepare incident information. During disasters such as floods, fires, earthquakes, and severe weather events, people may need to communicate their situation, identify their location, and share useful details about an incident. A mobile interface that brings these activities together can make the process easier to understand and use.

The application is designed around an SOS workflow, location access, incident reporting, and notification-related capabilities. Flutter provides the user interface framework, while Riverpod supports state management and GoRouter supports navigation between screens. Geolocator is included to access device location subject to user permission. Other packages provide a foundation for network requests, media selection, connectivity checks, local storage, and local notifications.

The project follows a modular structure so that features can be developed and tested separately. The current design is a foundation rather than a confirmed connection to public emergency infrastructure. A working emergency service requires secure backend processing, verified contacts, reliable delivery, privacy safeguards, and operational testing. The current build also requires compilation fixes before functional test results can be confirmed. Future work includes completing the build, validating each module, integrating an appropriate backend, and evaluating usability and reliability.

TABLE OF CONTENTS

SI.NO	TITLE	PAGE NO
1	ABSTRACT	3
2	INTRODUCTION	4
3	LITERATURE REVIEW	8
4	PROPOSED SYSTEM	10
4.1	Architecture Diagram	10
4.2	System Workflow	11
4.3	Functional Requirements	11
4.4	Non-Functional Requirements	12
4.5	Feasibility and Design Considerations	12
5	MODULES DESCRIPTION	13
5.1	Module 1 – SOS and Emergency Workflow	13
5.2	Module 2 – Location and Incident Reporting	14
6	IMPLEMENTATION AND RESULTS	16
6.1	Experimental Setup	16
6.2	Results	17
6.3	Test Plan	18
6.4	Limitations	19
7	CONCLUSION AND FUTURE WORK	20
8	REFERENCES	22

1. INTRODUCTION

1.1 Background

Disasters can affect people, homes, public infrastructure, transportation, and essential services. In an emergency, citizens may need to understand what action to take, communicate the nature of an incident, or provide information about where assistance is required. These activities can become difficult when people are under stress or when normal communication channels are disrupted.

Smartphones offer several capabilities that may assist in these situations. They can present simple instructions, access location information, capture images, and display notifications. A well-designed application can organize these capabilities into a clear workflow. However, a mobile application alone cannot guarantee that help will arrive. Reliable response also depends on network availability, accurate information, operational procedures, and connections to authorized responders.

ResQAI is conceived as a citizen-oriented mobile application that brings emergency-related interactions into a single interface. The project uses Flutter, which allows a shared application codebase to target supported platforms. Its modular design separates user interface behaviour, navigation, location access, and incident information so that each area can be improved independently.

1.2 Problem Statement

During an emergency, a person may have to switch between different applications or communication methods to describe an incident and share relevant information. This can create confusion and increase the effort required to prepare a clear report. Location details may also be difficult to communicate accurately, especially when a user does not know the address of the affected place.

The problem addressed by this project is the need for a simple mobile interface that organizes SOS-related actions, location access, and incident reporting. The application should make these actions understandable, request permissions appropriately, and provide clear feedback

to the user. It should also be designed so that future backend services can process reports safely.

1.3 Objectives

- To design a citizen-focused mobile interface for disaster-related activities.
- To provide an SOS workflow that is easy to locate and navigate.
- To use device location services when the user grants permission.
- To support the preparation of incident information, including selected media where available.
- To organize the application into maintainable modules using modern Flutter packages.
- To establish a foundation for future backend connectivity and verified emergency service integration.
- To test navigation, permission handling, error states, and core user workflows before claiming operational readiness.

1.4 Scope

The project scope covers the mobile application structure, user-facing emergency workflow, location-related functionality, incident information entry, and supporting components such as navigation and state management. The application can be expanded as requirements become clearer and as backend services are made available.

The current scope does not assume that the application is connected to police, ambulance, fire services, government disaster platforms, or a live dispatch centre. Such integrations require official interfaces, authorization, security controls, and testing with the relevant organizations. The project should be presented as a prototype until these integrations and the end-to-end workflow have been verified.

1.5 Significance of the Project

The project provides practical experience in building a mobile application around a socially relevant problem. It brings together user interface design, state management, routing, device permissions, location services, and potential network communication. It also highlights the

importance of designing software for stressful situations, where clear labels, predictable navigation, and useful error messages are especially important.

From a learning perspective, ResQAI demonstrates how independent components can be arranged into a larger application. It also encourages careful thinking about privacy, reliability, and the difference between a user-interface prototype and a fully operational emergency response service.

2. LITERATURE REVIEW

2.1 Mobile Technology in Disaster Management

Mobile technology is widely used for communication, navigation, information access, and coordination. During disasters, a mobile device may help a user view instructions, record observations, or communicate relevant information. Applications can provide a consistent interface for these actions, reducing the need for users to remember several different steps.

The value of an emergency application depends on more than its visual design. It must communicate its current state clearly, avoid misleading confirmations, and behave safely when network access or location permission is unavailable. Emergency-related software should also distinguish between preparing a report and successfully delivering that report to a response organization.

2.2 Location-Based Services

Location-based services use geographical information obtained from a device or another source. A location can be represented through coordinates and may be displayed on a map or included in an incident report. In a disaster context, location information may help describe where assistance is needed, particularly when the user cannot provide a complete street address.

Location data is not always exact. Indoor environments, weak satellite visibility, device settings, and network conditions can affect accuracy. Therefore, an application should handle denied permissions and unavailable readings, and it should avoid presenting an uncertain location as exact. The user should be able to understand when location access is being requested and why it is useful.

2.3 Incident Reporting and Notifications

Incident reporting allows users to describe an event through structured information such as a category, description, time, location, and optional image. Structured reports are easier to review than unorganized messages, provided that the form remains short enough for use

under pressure. Media can provide context, but uploading images may consume data and may expose private or sensitive information.

Notifications can draw attention to updates, reminders, or changes in an application's state. Local notifications are generated on the device, whereas remote notifications generally require a server-side service and appropriate delivery infrastructure. The project should not claim to provide live disaster alerts until a verified source and notification pipeline have been implemented and tested.

2.4 State Management and Navigation

A mobile application often contains multiple screens and shared data. State management helps coordinate the data shown by different widgets and reduces the need to pass information manually through many layers. Riverpod is included in this project as the state-management approach. Navigation allows users to move between screens and return to previous steps. GoRouter is included to organize these routes and transitions.

Separating state management from presentation can improve maintainability. For example, the user interface can display a loading state while a location request is in progress, then show the result or an error message. This separation makes it easier to test logic and adjust the interface without rewriting the entire application.

2.5 Research Gap and Project Motivation

The project is motivated by the opportunity to combine several emergency-related interactions in a single mobile application. Rather than treating location, incident details, and SOS actions as unrelated tasks, ResQAI aims to organize them into a consistent workflow. The goal at this stage is to establish a clear and extensible application structure.

A complete evaluation would require comparison with existing applications, user studies, and tests under realistic network and location conditions. These activities are outside the claims made in this report. The project therefore presents a design and implementation foundation, with validation and real-world integration identified as future work.

3. PROPOSED SYSTEM

The proposed system is a Flutter mobile application organized around emergency-related workflows. The user interacts with screens that guide them through SOS-related actions, location access, and incident information. The application uses separate packages for navigation, state management, location, media selection, connectivity, and network requests.

A modular approach allows the system to evolve gradually. The user interface can be developed and tested first; location and reporting behaviour can then be validated; and a backend can be added once the data format, security requirements, and authorized recipients are defined. This avoids presenting an unverified interface as a live emergency service.

3.1 ARCHITECTURE DIAGRAM

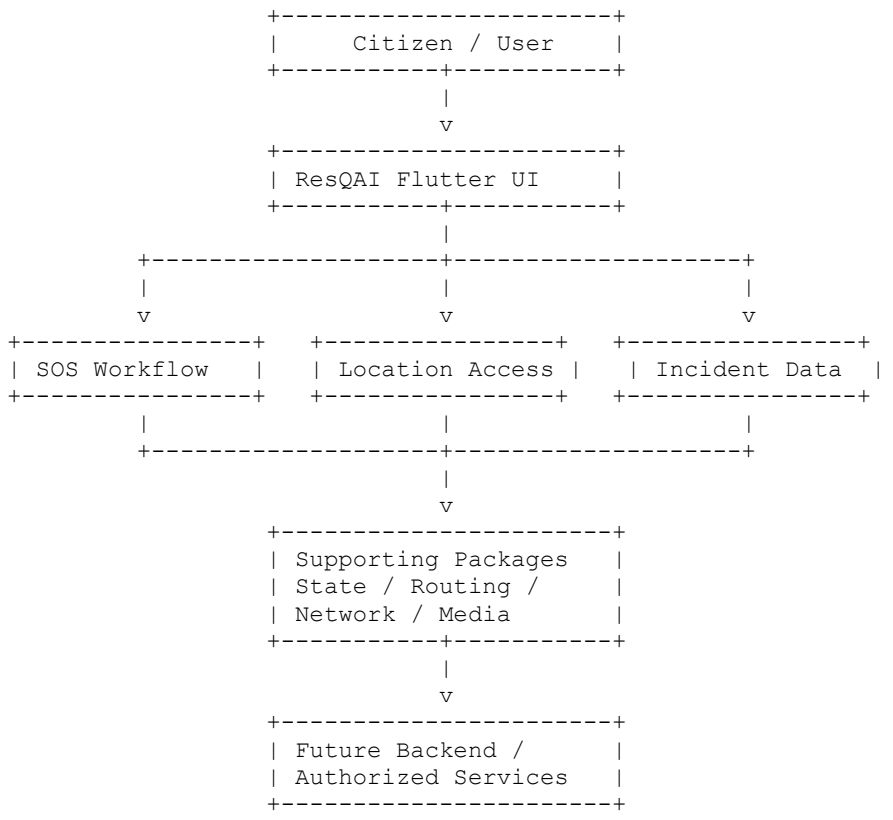


Figure 3.1: High-level architecture of the ResQAI application.

The diagram shows the intended organization of the application. The final backend and authorized emergency-service layer is a future integration point unless it is implemented and

verified separately. The diagram is conceptual and does not indicate that live alerts are currently being delivered.

3.2 System Workflow

A typical user workflow begins when the citizen opens the application and selects an emergency-related option. The application displays the relevant screen and guides the user through the available fields. If location is needed, the app requests permission and attempts to retrieve the device location. The user may then prepare incident details and select supporting media if the feature is available.

Before any report is submitted, the application should clearly show the information that will be shared. The application should validate required fields and provide a useful message when data is missing. If a network submission is added, the app should distinguish between a report prepared locally, a request sent to a server, and a server-confirmed submission. A message should never imply that emergency services have received a report unless the system has reliable confirmation.

3.3 Functional Requirements

- The application shall provide navigable screens for its main features.
- The application shall provide an SOS-related workflow with clear labels and feedback.
- The application shall request location permission before accessing protected location data.
- The application shall handle denied permissions and unavailable location readings gracefully.
- The application shall allow the user to prepare incident details and, where implemented, select media.
- The application shall display useful feedback for loading, success, and error states.
- The application should detect connectivity status where supported by the configured package.
- The application should be structured to support future backend communication.

3.4 Non-Functional Requirements

- Usability: important actions should be easy to identify and understand.
- Reliability: the interface should handle permission and network failures without crashing.
- Maintainability: modules should be separated so that changes can be made independently.
- Privacy: location and media should be accessed only when needed and with appropriate permission.
- Performance: screens should remain responsive and avoid unnecessary processing.
- Compatibility: the application should be tested on supported Flutter target devices.
- Transparency: the interface should accurately describe whether information has been saved or transmitted.

3.5 Feasibility and Design Considerations

Technical feasibility is supported by Flutter and its package ecosystem, which provide tools for building interfaces, managing state, navigating between screens, and accessing device features. The actual feasibility of live emergency communication depends on additional infrastructure and approvals that are not assumed in the current prototype.

Operational feasibility requires that the workflow be understandable to people with different levels of technical experience. Labels should use simple language, controls should be easy to reach, and critical actions should avoid ambiguous wording. The design should also account for poor connectivity and situations where users cannot grant location permission.

Security and privacy are essential design considerations. Location coordinates and incident images may reveal sensitive information. Any future server integration should use secure transport, validate input, control access, and define how long reports are retained. The app should collect only information necessary for its stated purpose.

4. MODULES DESCRIPTION

4.1 MODULE 1 – SOS AND EMERGENCY WORKFLOW

The SOS and emergency workflow module represents the main emergency-related interaction in ResQAI. Its purpose is to make the emergency action visible and guide the user through the next steps. A clear workflow is important because a user may be stressed, distracted, or working with limited time.

4.1.1 Main Responsibilities

- Present an identifiable entry point for the SOS workflow.
- Guide the user through the required information in a logical order.
- Display the current state of the workflow, such as editing, checking information, or showing an error.
- Provide navigation that allows the user to return to the previous screen where appropriate.
- Prevent misleading success messages when a request has not been confirmed by a service.

4.1.2 User Interaction

The user begins by opening the SOS screen. The interface should explain what the action does and which information may be requested. If the workflow uses location, the application should ask for the required permission and explain what happens if permission is denied. The user should be given an opportunity to review the incident details before any future submission feature sends them elsewhere.

A confirmation step may be appropriate for actions that could trigger an external request. However, confirmation should not make the workflow unnecessarily complicated. The interface should balance speed with preventing accidental activation. The exact interaction should be evaluated with users and adjusted based on test results.

4.1.3 Error Handling

The module should handle common failure cases, including missing required information, denied permissions, unavailable location readings, and network errors if online submission is implemented. Error messages should explain the problem in plain language and offer a useful

next step. For example, if location is unavailable, the interface could ask the user to enable location or enter a description manually, if that option is supported.

4.1.4 Safety Considerations

The presence of an SOS button does not by itself create a functioning emergency service. A live service requires a defined destination, secure message delivery, reliable acknowledgement, monitoring, and a procedure for handling reports. Until these elements are implemented and tested, the report should describe this module as an SOS workflow or prototype rather than a confirmed emergency dispatch system.

4.2 MODULE 2 – LOCATION AND INCIDENT REPORTING

The location and incident reporting module is intended to help users prepare information about an event. Location can provide a useful reference point, while a short description and optional image can communicate the nature of the incident. The design should make it clear which fields are required and which are optional.

4.2.1 Location Retrieval

The application includes the Geolocator package to support location access. The device may ask the user to grant permission before providing location data. If permission is denied, the application should not repeatedly request it without explanation. It should show a clear message and allow the user to continue with any alternative workflow supported by the application.

Location readings can vary in accuracy. A robust implementation should check whether a reading is available and should avoid assuming that every coordinate is exact. If the application displays coordinates or uses them in a report, the user should understand that location access depends on device settings and environmental conditions.

4.2.2 Incident Information

Incident information may include a category, description, time, location, and optional image. These fields can help organize the report and make later review easier. The interface should

avoid asking for unnecessary personal information. Text fields should have clear labels and sensible limits so that the report remains readable and useful.

4.2.3 Media Selection

The project includes an image-selection package, which can provide a way to attach visual information to an incident report. Media selection should be optional where possible, because the user may be unable to capture or upload an image during an emergency. The application should explain when an image has been selected and should not imply that the image has been uploaded unless upload functionality is implemented and confirmed.

4.2.4 Privacy and Data Handling

Location and images can reveal a user's movements, surroundings, or personal circumstances. Future storage and sharing features should therefore define who can view a report, how it is transmitted, and how long it is retained. Sensitive data should not be included in diagnostic logs unnecessarily. Permission requests should be tied to a clear feature and should explain the reason for access.

4.3 SUPPORTING COMPONENTS

In addition to the two main modules, the application uses supporting components to keep the code organized. Riverpod is included for state management, and GoRouter is included for navigation. Dio provides a client for future or existing HTTP requests. Connectivity monitoring can help the application recognize network state, although network connectivity alone does not prove that a server is reachable.

Shared preferences can support storage of simple local settings. Local notification functionality can be used for notifications generated on the device. Speech-to-text can provide an alternative input method if it is connected to the user interface and tested. The existence of a package dependency does not necessarily mean that every related feature is fully implemented; each component must be verified in the application before being reported as complete.

5. IMPLEMENTATION AND RESULTS

The implementation is organized as a Flutter project written in Dart. Flutter provides the widget-based user interface, while Dart contains the application logic. The project includes platform-specific folders and a collection of source files grouped by application responsibility. This structure supports the separation of screens, shared components, and core configuration.

5.1 EXPERIMENTAL SETUP

5.1.1 Hardware and Software

The development environment uses a MacBook with Visual Studio Code and the Flutter toolchain. Android build tools are used to compile the Android version of the application. A compatible Android emulator or physical Android device is intended for runtime testing. The exact SDK and package versions should be recorded from the final working environment because versions can affect build compatibility.

5.1.2 Technologies Used

- Flutter: framework for building the mobile user interface.
- Dart: programming language used for application logic.
- Riverpod: state-management library included in the project.
- GoRouter: routing and navigation package included in the project.
- Geolocator: package for requesting and retrieving device location.
- Dio: HTTP client package for network communication.
- Image Picker: package for selecting images from supported device sources.
- Connectivity Plus: package for observing network connectivity changes.
- Shared Preferences: package for simple local key-value storage.
- Flutter Local Notifications: package for local device notifications.

These technologies are listed based on the project dependencies. The final report should distinguish packages that are actively used by working features from packages that are present for planned functionality.

5.1.3 Development Procedure

The development procedure begins with creating the Flutter project structure and adding the required dependencies. The next step is to define the main screens and navigation paths. State management is then connected to the relevant screens so that the interface can reflect changes in the workflow. Device-specific functionality, such as location and media selection, requires permission handling and testing on an emulator or physical device.

After the core screens are connected, the project should be built and tested incrementally. Compilation errors should be fixed before runtime behaviour is evaluated. Each module should be tested independently first, followed by a basic end-to-end walkthrough. Finally, screenshots can be captured from the working application and included in the report.

5.2 RESULTS

At the time of preparing this report, the project structure and selected dependencies are in place, but a successful application build and complete functional test run have not yet been confirmed. Compilation issues have been identified in the theme configuration and SOS workflow navigation. These issues must be resolved before the application can be evaluated as a running prototype.

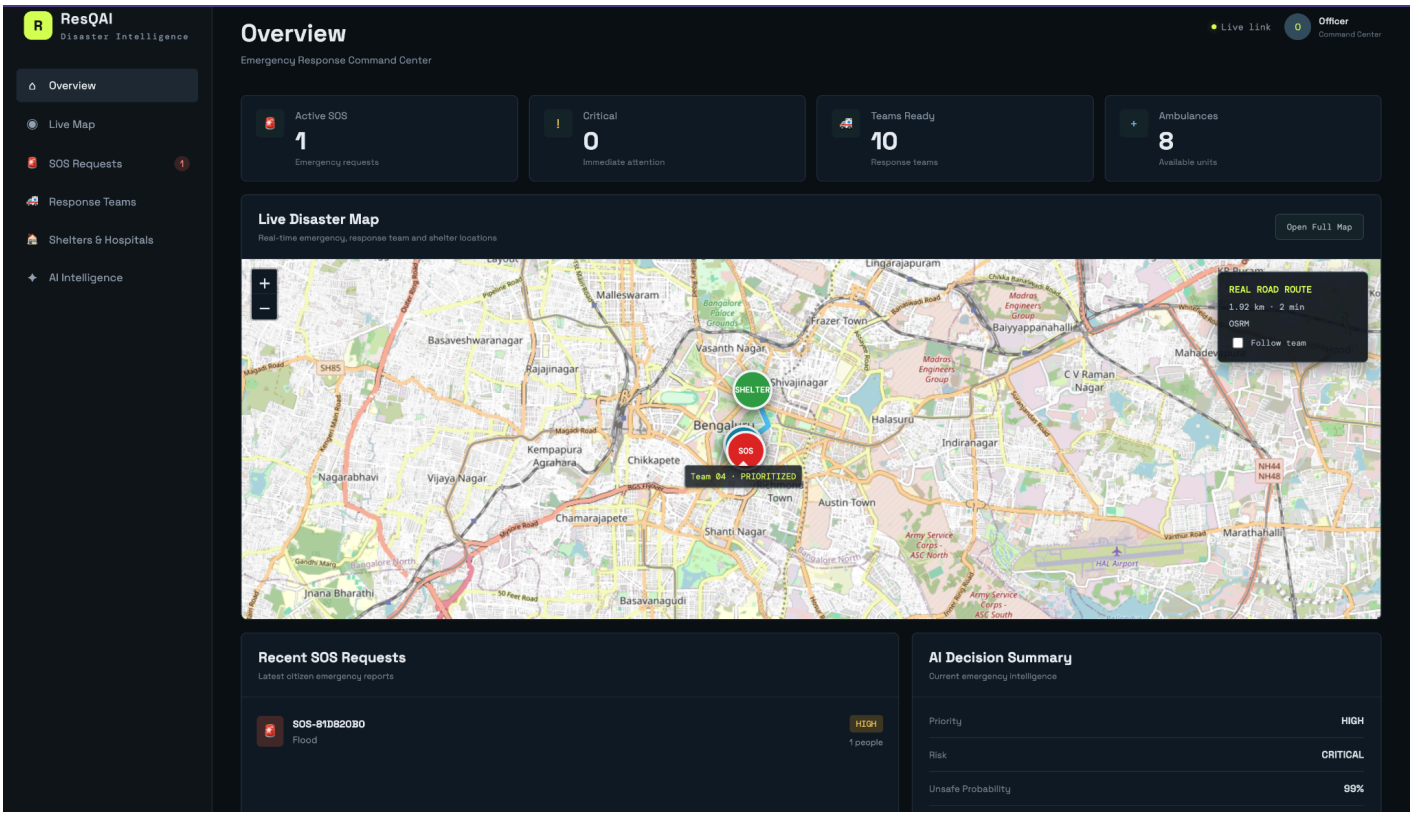
For this reason, this section does not claim measured performance, successful emergency delivery, or verified end-to-end operation. The expected outcomes below describe what should be checked after the build succeeds. They should be replaced or supplemented with actual observations and screenshots from the final application.

5.2.1 Expected Outcomes to Verify

- The application opens without a crash and displays the intended initial screen.
- Navigation between the main screens works as expected.
- The SOS workflow can be opened and completed through its supported local steps.
- Location permission is requested appropriately and denied-permission states are handled.
- Incident information can be entered and validated according to the implemented form.
- Media selection works on the target device if the feature is connected to the interface.

- Network and connectivity errors are shown clearly if online functions are enabled.

5.2.2 Screenshots



5.3 TEST PLAN

The following test plan can be used to record the results after compilation errors have been resolved. The status column should be filled with the actual result obtained during testing.

Test ID	Test Case	Expected Result	Status
T01	Launch application	Initial screen opens without a crash	Pending
T02	Open SOS screen	SOS workflow screen is displayed	Pending
T03	Navigate back	User returns to the previous screen	Pending
T04	Grant location permission	Location request can proceed	Pending
T05	Deny location permission	Clear message or alternative path is shown	Pending
T06	Submit incomplete incident form	Missing information is identified	Pending
T07	Select incident image	Selected image is displayed if implemented	Pending
T08	Disable network	Network-dependent feature handles failure	Pending

T09	Build Android application	Build completes without compilation errors	Pending

Table 5.1: Functional test cases for the ResQAI prototype.

Testing should be performed in a controlled environment. Emergency workflows should not be tested by contacting real emergency services unless an approved test procedure and authorized test destination are available.

5.4 LIMITATIONS

The main limitation is that the application build and end-to-end behaviour remain unverified. The presence of packages for networking, notifications, location, or media does not establish that all features are connected or functioning. The current project should therefore be described as a prototype under development.

Other limitations may include dependence on device permissions, variable location accuracy, network availability, and the absence of a verified backend. A production disaster response service would also require authentication, secure data handling, server-side validation, reliable delivery acknowledgement, monitoring, and a clear operational process for responders.

The application should not be relied upon as the sole method of obtaining emergency assistance until it has been completed, independently tested, and formally connected to appropriate services.

6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

ResQAI is a citizen-focused disaster response mobile application project built around Flutter and Dart. It aims to bring SOS-related workflows, location access, and incident information into a single interface. The project uses a modular structure and packages for navigation, state management, location, media selection, connectivity, and network communication.

The project also demonstrates that emergency-related software must be designed with particular attention to clarity, reliability, permissions, privacy, and honest status messages. A user interface can support the preparation of information, but a complete response service requires reliable backend processing and an authorized connection to emergency responders.

The application structure and selected dependencies provide a basis for continued development. However, compilation errors and functional testing must be completed before successful execution can be claimed. The report therefore records the current state transparently and identifies the validation work that remains.

6.2 Future Work

6.2.1 Resolve Build and Navigation Issues

The immediate next step is to correct the theme type mismatch and the SOS workflow navigation error, then rebuild the application. Once it compiles, the app should be launched on a supported emulator or physical device and the main screens should be checked for runtime errors.

6.2.2 Backend and Database Integration

A backend can be introduced to store incident reports and provide controlled access to authorized reviewers. The backend design should define data fields, authentication, access control, retention rules, and error responses. The application should clearly distinguish local data from data that has been successfully stored on a server.

6.2.3 Verified Emergency Service Integration

Future integration with emergency services must use authorized channels and an agreed operating procedure. The system should provide delivery acknowledgements, audit logs, and safeguards against accidental or repeated reports. Any demonstration should use a test endpoint rather than contacting live emergency services without permission.

6.2.4 Maps and Incident Visualization

A map interface could display the user's location and, where appropriate, reports that the user is authorized to view. Any map provider must be configured according to its terms and attribution requirements. Location information should be displayed with appropriate accuracy indicators and privacy controls.

6.2.5 Multilingual and Accessible Interface

The application could support multiple languages and accessible text, contrast, and touch targets. Simple instructions and consistent icons may help people who are unfamiliar with technical applications. Voice input may be explored as an alternative input method, but it should be tested in noisy environments and should not be treated as a replacement for clear manual controls.

6.2.6 Offline Support and Reliability

Disasters may interrupt internet access. Future versions could allow users to prepare a report locally and submit it when a connection becomes available, provided that the interface clearly communicates the report's status. The design should prevent duplicate submissions and protect locally stored sensitive information.

6.2.7 Security and Evaluation

Future work should include a privacy review, security testing, and usability evaluation with representative users. Testing should cover low connectivity, permission denial, inaccurate location, interrupted requests, and invalid input. The project can then report actual success rates and user feedback rather than relying on assumptions.

7. REFERENCES

Dart Team. (n.d.). Dart documentation. https://dart.dev/

Flutter Community. (n.d.). Connectivity Plus package. pub.dev. https://pub.dev/packages/connectivity_plus

Flutter Community. (n.d.). Dio package. pub.dev. https://pub.dev/packages/dio

Flutter Community. (n.d.). Geolocator package. pub.dev. https://pub.dev/packages/geolocator

Flutter Community. (n.d.). GoRouter package. pub.dev. https://pub.dev/packages/go_router

Flutter Community. (n.d.). Image Picker package. pub.dev. https://pub.dev/packages/image_picker

Flutter Community. (n.d.). Shared Preferences package. pub.dev. https://pub.dev/packages/shared_preferences

Flutter Team. (n.d.). Flutter documentation. https://docs.flutter.dev/

Riverpod. (n.d.). Riverpod documentation. https://riverpod.dev/

United Nations Office for Disaster Risk Reduction. (n.d.). Disaster risk reduction terminology and resources. https://www.undrr.org/terminology

World Health Organization. (n.d.). Health emergency and disaster risk management framework. https://www.who.int/